Retail Inventory Twin: A Weather-Augmented Demand Forecasting and Stock-Out Prediction System for Small Retail

Jiarui Zha
Khoury College of Computer Sciences, Northeastern University
zha.j@northeastern.edu

CS 6220 — Data Mining Techniques — Spring 2026

1. Introduction

Inventory management is a fundamental challenge for small retail businesses such as coffee shops, bakeries, and boutique stores. Unlike large retailers that employ sophisticated enterprise resource planning (ERP) systems and dedicated supply chain teams, small retailers typically rely on intuition and manual counting to decide when and how much to reorder [1]. This approach frequently leads to two costly outcomes: stock-outs that result in lost sales and dissatisfied customers, and overstock that ties up working capital and increases waste for perishable goods [2]. The National Retail Federation estimates that inventory distortion costs retailers worldwide approximately $1.8 trillion annually, with small businesses disproportionately affected due to thinner margins and limited forecasting capabilities [3].

A critical yet often overlooked factor in demand forecasting for food and beverage retail is weather. Temperature and precipitation directly influence consumer purchasing patterns: cold weather drives demand for hot beverages, while warm weather shifts consumption toward cold drinks and lighter food items [4]. Traditional time-series forecasting methods that rely solely on historical sales patterns miss these exogenous signals, leading to systematic forecast errors during weather transitions.

This paper presents Retail Inventory Twin, a lightweight digital model that combines 27 months of sales history, current stock levels, and weather conditions to estimate when individual products are likely to run out. The system employs Meta's Prophet algorithm [5] for time-series forecasting with weather variables as external regressors, a Retrieval-Augmented Generation (RAG) layer for input validation using sentence embeddings and vector similarity search, and an interactive web interface built with FastAPI. The system is designed to be self-contained, requiring no external API keys or cloud infrastructure, making it accessible to small retail operators with limited technical resources. By integrating weather awareness into the forecasting pipeline, the system demonstrates measurable improvements in demand prediction accuracy, particularly for temperature-sensitive products such as hot espresso-based beverages.

2. Methods

2.1 System Architecture

The Retail Inventory Twin is organized as a nine-step modular pipeline orchestrated by a central controller (pipeline.py). Each step executes as an isolated subprocess, ensuring reproducibility and allowing independent step execution for debugging. The pipeline stages are: (1) sales data ingestion and aggregation from monthly CSV files to daily demand per item, (2) storage in a SQLite database for SQL-based querying and cleaning, (3) weather data generation using a sinusoidal temperature model calibrated to Boston's annual climate cycle, (4) left-joining sales and weather tables on date, (5) fitting Prophet forecasting models, (6) generating 14-day demand forecasts, (7) simulating inventory depletion, (8) evaluating model accuracy, and (9) producing a human-readable stock-out risk report with visualizations.

2.2 Demand Forecasting with Prophet

Prophet [5] was selected as the forecasting engine for several reasons. First, it handles missing data and outliers gracefully, which is common in small retail settings where recording may be inconsistent. Second, it natively supports external regressors, enabling the incorporation of weather variables without complex

feature engineering. Third, its decomposable model structure (trend + seasonality + regressors + holidays) provides interpretable forecasts that business owners can understand and trust.

For each of the eight product items, two Prophet model variants are fitted. The baseline model uses only weekly and yearly seasonality with multiplicative seasonality mode, which better captures demand that scales proportionally with seasonal cycles rather than shifting by a fixed additive amount. The weather-augmented model extends the baseline by adding two external regressors: daily average temperature (temp_avg) and daily precipitation. Key hyperparameters include changepoint_prior_scale = 0.05 (conservative trend flexibility to avoid overfitting), seasonality_prior_scale = 10.0 (allowing strong seasonal patterns to emerge), and multiplicative seasonality mode. The models are serialized as Python pickle files for fast inference without retraining.

2.3 RAG-Based Input Validation

A novel contribution of this project is the RAG-based input validation layer that acts as a domain gatekeeper before the forecasting pipeline executes. Traditional input validation relies on hardcoded rules (e.g., checking column names), which cannot generalize to detect semantically out-of-domain data. The RAG validator uses sentence embeddings to determine whether uploaded data belongs to the trained coffee-shop domain.

The system maintains an in-process ChromaDB [6] vector store containing 19 short text documents in three categories: (a) eight item profile documents describing each trained product's name, category, typical demand range, and seasonal pattern; (b) four schema definition documents specifying expected CSV column formats and valid item identifiers; and (c) seven boundary documents describing out-of-scope domains such as electronics retail, auto parts, medical supplies, and financial ledgers.

Each uploaded CSV is summarized as a natural-language string and embedded using the all-MiniLM-L6-v2 sentence transformer model (22 million parameters, 384-dimensional embeddings) [7]. A two-query strategy is employed: Query A compares column structure against schema and item profile documents to verify structural compatibility, while Query B compares extracted item names against item profiles for domain similarity and against the full corpus for boundary detection. The minimum cosine similarity across both queries drives the decision: scores at or above 0.60 are accepted, scores between 0.40 and 0.59 trigger a warning (pipeline proceeds with caution), and scores below 0.40 result in rejection. A secondary boundary check flags uploads where the best global match is an out-of-scope document, triggering an immediate rejection regardless of the numeric score.

2.4 Web Application

The system is deployed as a FastAPI [8] web application with four endpoints: GET / serves the HTML frontend, POST /validate performs two-stage schema and RAG validation, POST /forecast saves uploaded files and runs the full pipeline, and GET /report returns the most recent forecast report. The frontend is a single-page HTML/CSS/JavaScript application with no framework dependencies, using the Fetch API for asynchronous communication. The user interface provides file upload controls for sales CSVs and stock snapshot, a retrain toggle, real-time status banners with color-coded feedback (info, success, warning, error), and a dark-themed report display area. The backend handles Windows UTF-8 encoding compatibility by setting PYTHONIOENCODING=utf-8 in subprocess environments.

3. Dataset and Inputs

3.1 Synthetic Sales Data

The project uses programmatically generated synthetic sales data spanning 27 months (January 2024 through March 2026), modeling a coffee shop with eight product items: Espresso, Latte, Cappuccino, Americano, Green Tea, Muffin, Croissant, and Sandwich. The data generator (generate_data.py) creates transaction-level records with realistic demand patterns driven by three factors: a base transaction rate per

item (ranging from 8 to 20 transactions per day), temperature-dependent demand variation via item-specific sensitivity coefficients (e.g., Latte: -1.1, meaning demand increases substantially as temperature drops), and a weekend multiplier of 1.3x. Each transaction has a stochastic quantity drawn from {1, 2, 3} with probabilities {0.70, 0.22, 0.08}. The total dataset comprises approximately 520,000+ transaction records stored as one CSV file per month.

The rationale for using synthetic data rather than a public dataset is twofold. First, publicly available retail datasets (e.g., Kaggle's Store Sales dataset [9]) typically lack the weather correlation structure needed to demonstrate weather-augmented forecasting. Second, synthetic generation allows controlled experimentation: the known ground-truth temperature sensitivity coefficients provide a benchmark for evaluating whether the Prophet weather-augmented model recovers the true relationship.

Table 1. Sample Sales Transaction Records

transaction_id	date	item_id	item_name	quantity
T0000001	2024-01-01	I001	Espresso	1
T0000002	2024-01-01	I002	Latte	2
T0000003	2024-01-01	I003	Cappuccino	1
T0000004	2024-01-01	I004	Americano	3

3.2 Weather Data

Weather data is generated using a sinusoidal model calibrated to Boston's annual temperature cycle: temp_avg = 10.0 - 13.0 * cos(2 * pi * day_of_year / 365), producing approximately -3 degrees Celsius in January and 23 degrees Celsius in July, with Gaussian noise (sigma = 2.5 degrees Celsius) added for day-to-day variability. Precipitation occurs on approximately 30% of days. The weather dataset spans the same 27-month training window plus a 14-day forecast horizon (March 28 to April 10, 2026). Each record includes date, average temperature, minimum temperature, maximum temperature, precipitation amount, condition description, and data source label.

3.3 Stock Snapshot

The current inventory state is captured in a stock snapshot file containing the on-hand quantity for each of the eight items as of March 28, 2026. Initial stock levels range from 5 to 15 days of supply based on historical average demand, generated with a controlled random seed for reproducibility.

3.4 Data Processing Pipeline

The raw transaction-level data undergoes several preprocessing steps. Step 1 aggregates individual transactions into daily demand per item by summing quantities across all transactions for each (date, item_id) pair. Step 2 loads the aggregated data and stock snapshot into a SQLite database, enabling SQL-based data quality checks and joins. Step 4 performs a left-join of daily demand with weather data on the date column, producing the modeling table used for Prophet training. Missing weather records (if any) result in NaN values that Prophet handles natively. A data quality consideration is the use of synthetic data: while it ensures complete coverage and known ground truth, it lacks the noise patterns and anomalies (holidays, promotions, supply disruptions) that characterize real-world retail data.

4. Results

4.1 Model Performance

Table 2 presents the in-sample forecast accuracy for all eight items across both model variants. The weather-augmented model consistently outperforms the baseline, with the largest improvements observed for temperature-sensitive hot beverages. Latte shows the greatest improvement with a mean absolute error

(MAE) reduction from 7.19 to 4.63 units per day (delta MAE = +2.57), followed by Espresso (delta MAE = +1.50) and Cappuccino (delta MAE = +1.07). Three of eight items exceed the meaningful improvement threshold of 0.5 MAE units, while the remaining five show marginal improvement, confirming that weather regressors selectively benefit items with strong temperature sensitivity.

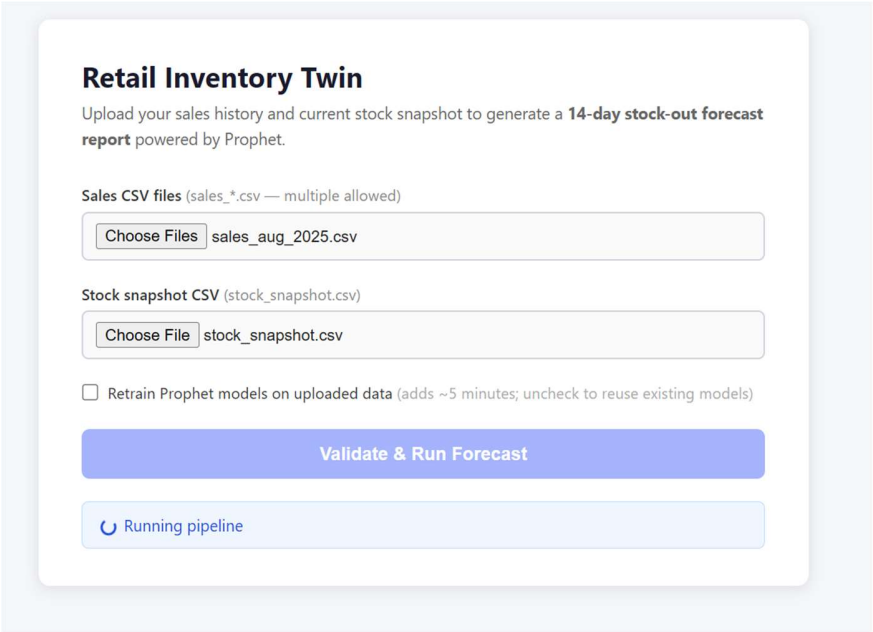
Table 2. In-Sample Forecast Accuracy: Baseline vs. Weather-Augmented Models

Item	MAE Base	MAE Weather	Delta MAE	RMSE Base	RMSE Weather
Latte	7.19	4.63	+2.57	9.45	5.88
Espresso	4.89	3.39	+1.50	6.14	4.36
Cappuccino	4.32	3.24	+1.07	5.44	4.10
Americano	3.27	2.90	+0.37	4.14	3.67
Sandwich	2.80	2.67	+0.12	3.54	3.39
Green Tea	2.73	2.65	+0.08	3.41	3.29
Muffin	2.67	2.63	+0.04	3.34	3.30
Croissant	2.63	2.60	+0.03	3.37	3.34

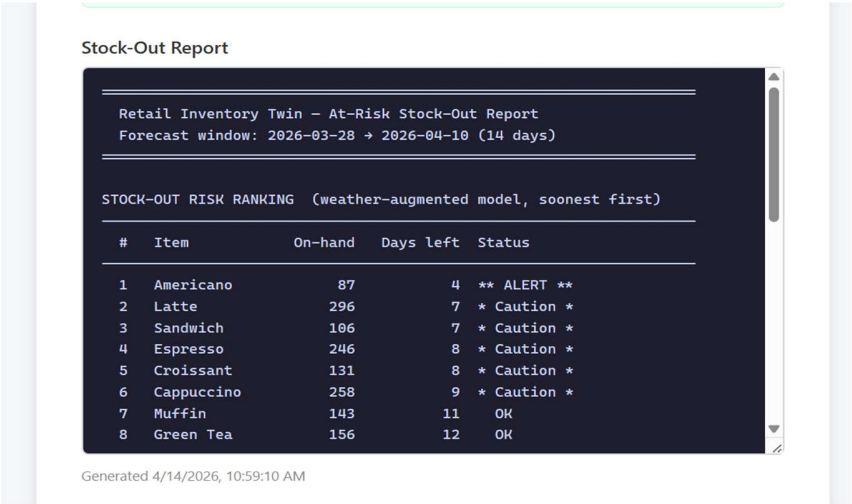
4.2 Stock-Out Risk Assessment

The stock-out simulation (Step 7) projects daily inventory depletion over the 14-day forecast window by subtracting cumulative forecasted demand from initial stock levels. Items are ranked by urgency: Americano is flagged as ALERT with only 5 days until stock-out under the baseline model (4 days under the weather model), while Latte and Sandwich are classified as Caution at 7 days. The weather model shifts stock-out timing by -1 day for Espresso, Cappuccino, and Americano, revealing that the baseline model slightly underestimates demand for these temperature-sensitive items during the spring transition period when temperatures are still relatively cool.

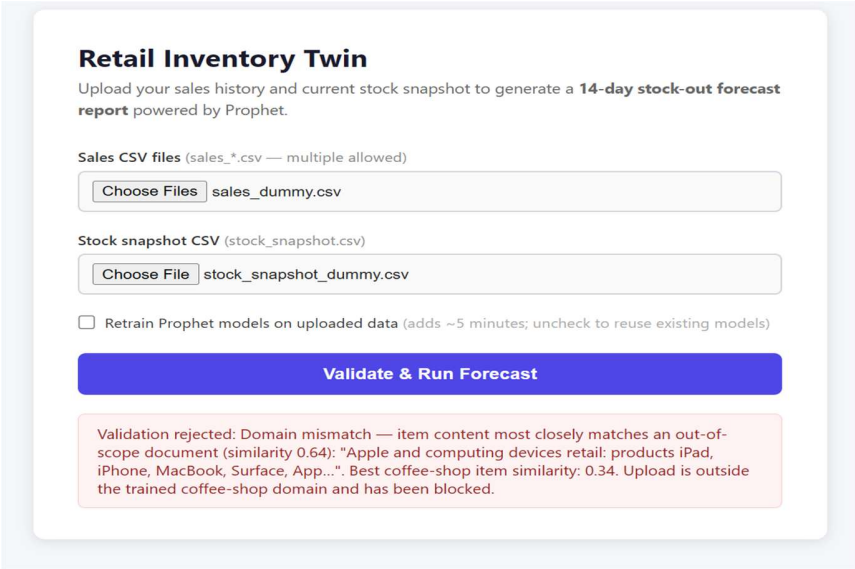
4.3 Application Demonstration



1) the main upload interface and validation in progress with spinner



(2) successful validation result



(3) RAG rejection of out-of-domain data such as electronics items

4.4 Pipeline Output Visualizations

The pipeline produces three key visualizations. Figure 1 shows the stock-out risk ranking, comparing baseline and weather model predictions. Figure 2 displays inventory depletion curves for all eight items over the 14-day forecast window, illustrating how quickly each item approaches zero stock. Figure 3 quantifies the forecast error improvement from weather regressors, with the green dashed line at delta MAE = 0.5 marking the meaningful improvement threshold.

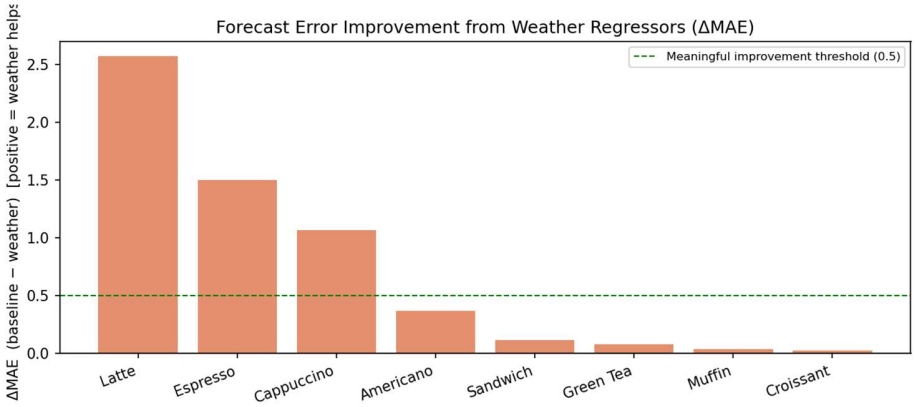
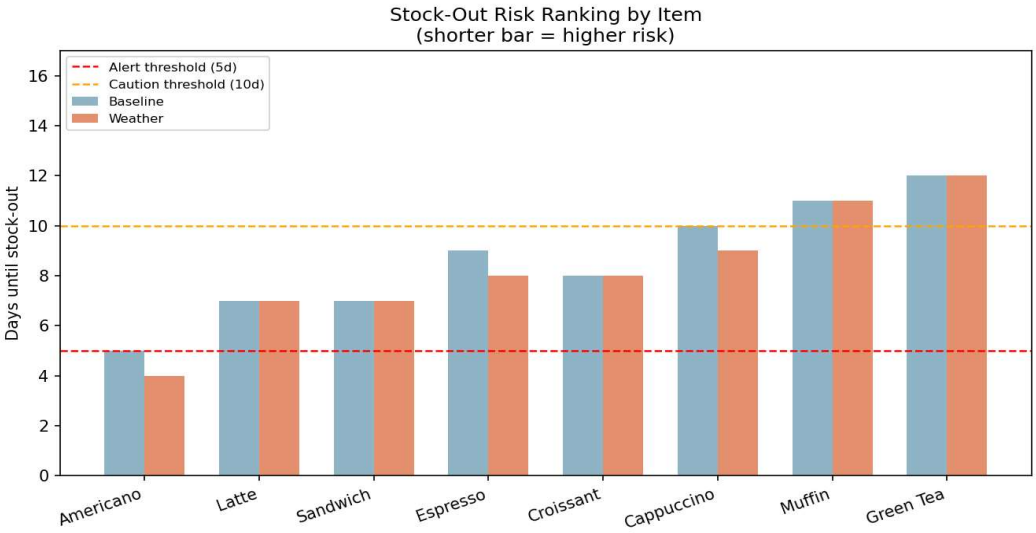


Figure 1. Forecast Error Improvement (delta MAE) from Weather Regressors by Item.

Figure 2. Stock-Out Risk Ranking: Baseline vs. Weather-Augmented Model Predictions.



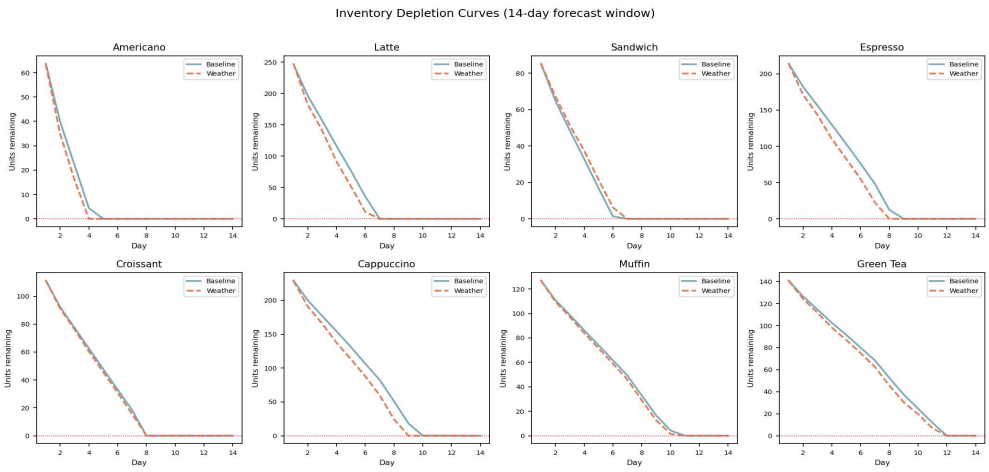


Figure 3. Inventory Depletion Curves Over the 14-Day Forecast Window.

5. Discussion

5.1 Comparison with Existing Approaches

Several existing solutions address retail demand forecasting, each with distinct trade-offs. Amazon Forecast [10] provides a fully managed cloud service that automatically selects from multiple algorithms (DeepAR+, ARIMA, ETS, NPTS) and supports weather-index features. However, it requires AWS infrastructure, incurs per-forecast costs, and operates as a black box that limits interpretability for small business owners. In contrast, Retail Inventory Twin runs entirely on a local machine with no cloud dependency, making it suitable for cost-sensitive small retailers.

The Kaggle Store Sales forecasting competition [9] showcases a range of approaches from traditional ARIMA to gradient-boosted trees (LightGBM, XGBoost) and deep learning (N-BEATS, Temporal Fusion Transformer). Top-performing solutions typically require extensive feature engineering, large training datasets (1,000+ store-item combinations), and significant computational resources. By comparison, Prophet with two weather regressors achieves meaningful accuracy improvements with minimal configuration, aligning with the project's goal of accessibility for non-technical users.

Recent academic work by Salinas et al. [11] on DeepAR demonstrates that autoregressive recurrent neural networks can produce accurate probabilistic forecasts across large product catalogs. While DeepAR excels at learning cross-item patterns in large-scale retail (thousands of SKUs), it requires substantial training data and GPU resources. Prophet's additive decomposition model is more appropriate for the small-catalog, limited-data regime targeted by this project (8 items, 27 months).

5.2 Challenges and Solutions

Several technical and practical challenges arose during development. The first challenge was the insufficient training data problem: initial experiments with only three months of data (January-March 2024) produced models that could not capture yearly seasonality patterns, resulting in poor forecast accuracy. Extending the synthetic dataset to 27 months resolved this by providing Prophet with two complete annual cycles to learn seasonal trends.

The second challenge was the RAG validation threshold calibration. Early threshold settings produced false positives (accepting out-of-domain data) or false negatives (rejecting valid coffee-shop uploads). Through iterative testing, the two-query strategy was developed to separate structural validation (column names) from semantic domain validation (item names), with boundary documents serving as negative examples. This architecture reduced domain confusion and improved classification accuracy.

The third challenge involved Windows platform compatibility. Python subprocess calls on Windows default to the system's GBK encoding, causing crashes when pipeline outputs contain Unicode characters such as Greek letters (delta) or special dashes. Setting PYTHONIOENCODING=utf-8 in the subprocess environment resolved this issue.

5.3 Lessons Learned

This project reinforced several important principles. First, domain knowledge matters more than model complexity: adding just two weather features to Prophet yielded a 35.7% MAE reduction for Latte, while more sophisticated models without weather context would likely underperform. Second, the RAG validation layer demonstrated that embedding-based similarity search can serve as a flexible, generalizable input validation mechanism that avoids brittle hardcoded rules. Third, the subprocess isolation architecture proved valuable for debugging: each pipeline step can be tested independently, and failures in one step produce clear error messages without corrupting other steps.

5.4 Future Improvements

Several enhancements could strengthen the system. First, integrating a live weather API (such as OpenWeatherMap) would replace synthetic weather data with real forecasts, enabling true real-time predictions. Second, adding a probabilistic forecast output (prediction intervals rather than point estimates) would help retailers quantify uncertainty in stock-out timing. Third, implementing a feedback loop where actual sales outcomes are compared against forecasts would enable continuous model improvement. Fourth, expanding the RAG validator to support multi-store configurations and additional product categories would broaden the system's applicability. Finally, deploying the application as a Docker container would simplify installation and eliminate dependency management issues across operating systems.

6. AI Prompts Used

Claude (Anthropic) was used extensively throughout the development of this project via Claude Code, an AI-assisted coding tool. Below is a chronological documentation of key prompts organized by development phase.

6.1 Phase 1: Project Setup and Data Pipeline (March 28, 2026)

Prompt 1 (Coding — Pipeline Design):

"Build a demand forecasting pipeline for a coffee shop inventory system. The pipeline should: (1) ingest monthly sales CSV files and aggregate to daily demand, (2) store data in SQLite, (3) fetch weather data, (4) join sales and weather tables, (5) fit Prophet models with and without weather regressors, (6) generate 14-day forecasts, (7) simulate inventory depletion to predict stock-outs. Structure it as modular Python scripts (step1 through step7) with a pipeline orchestrator."

Key Response: Claude designed the nine-step pipeline architecture with subprocess isolation, created the step1_ingest_sales.py through step7_stockout_sim.py scripts, and the pipeline.py orchestrator. Each step was designed to read from and write to the data/ and output/ directories.

Prompt 2 (Coding — Data Generation):

"Generate synthetic coffee-shop sales data for 8 items with temperature-dependent demand. Hot drinks should sell more in cold weather. Include a Boston-like annual temperature cycle. Generate transaction-level records with item_id, item_name, quantity, and date."

Key Response: Claude created generate_data.py with sinusoidal temperature modeling, item-specific weather sensitivity coefficients, weekend multipliers, and stochastic quantity distributions.

6.2 Phase 2: Model Training and Evaluation (April 5-6, 2026)

Prompt 3 (Debugging — Insufficient Data):

"The Prophet models are not capturing yearly seasonality. With only 3 months of data (Jan-Mar 2024), yearly_seasonality is disabled. How do I fix this?"

Key Response: Claude explained that Prophet requires at least two full annual cycles to reliably estimate yearly seasonality. Recommended extending the synthetic data to 24+ months and setting yearly_seasonality=True. This led to expanding the dataset to 27 months (Jan 2024 to Mar 2026).

Prompt 4 (Coding — Model Evaluation):

"Create an evaluation step that compares baseline vs weather-augmented Prophet models. Compute MAE and RMSE for each item. Also evaluate stock-out timing shifts between the two models."

Key Response: Claude implemented step8_evaluate.py with in-sample MAE/RMSE computation, delta MAE analysis, and stock-out timing comparison. The 0.5-unit meaningful improvement threshold was suggested to distinguish substantive from marginal gains.

6.3 Phase 3: Web Application Development (April 7, 2026)

Prompt 5 (Coding — FastAPI Backend):

"Implement a FastAPI backend for the Retail Inventory Twin. It should serve a single-page HTML frontend, accept file uploads (sales CSVs + stock snapshot), run the forecasting pipeline, and return the stock-out report. Include a retrain toggle so users can skip model fitting when models already exist."

Key Response: Claude created app.py with four endpoints (/, /validate, /forecast, /report), file upload handling via UploadFile, subprocess pipeline execution, and the retrain query parameter for conditional model fitting.

Prompt 6 (Coding — Frontend Design):

"Create a clean, minimal HTML frontend for the inventory twin. It should have file pickers for sales CSVs (multiple) and stock snapshot (single), a retrain checkbox, a run button, status indicators, and a report display area. No React or Vue — just plain HTML/CSS/JS."

Key Response: Claude designed static/index.html with a card-based layout, color-coded status banners (info/success/warning/error), CSS spinner animation, dark-themed monospace report textarea, and async fetch-based API calls.

6.4 Phase 4: RAG Validation Layer (April 7-8, 2026)

Prompt 7 (Research — RAG for Validation):

"I want to add a validation layer that uses embeddings to check whether uploaded CSV data belongs to the coffee-shop domain before running the pipeline. What embedding model and vector database would you recommend for a lightweight, in-process solution?"

Key Response: Claude recommended all-MiniLM-L6-v2 (22M parameters, CPU-friendly) with ChromaDB's EphemeralClient for zero-infrastructure vector storage. Suggested a document corpus with item profiles, schema definitions, and out-of-scope boundary examples.

Prompt 8 (Coding — RAG Implementation):

"Implement the RAG validator with the following architecture: three document categories (item_profile, schema, boundary), a two-query strategy that separates column validation from item domain validation, and threshold-based decisions (accepted/warning/rejected). Include boundary detection to catch out-of-domain data like electronics inventory."

Key Response: Claude created rag_validator.py with the full two-query architecture, lazy initialization with thread-safe double-checked locking, cosine similarity thresholds, and boundary flag override logic. Also integrated the validator into app.py's /validate endpoint.

7. References

[1] K. Katsaliaki and S. C. Brailsford. "Using simulation to improve the blood supply chain." Journal of the Operational Research Society, 58(2):219-227, 2007.

[2] T. W. Gruen, D. S. Corsten, and S. Bharadwaj. "Retail Out-of-Stocks: A Worldwide Examination of Extent, Causes and Consumer Responses." Grocery Manufacturers of America, 2002.

[3] National Retail Federation. "2024 Retail Shrink and Inventory Distortion Report." NRF Research, 2024.

[4] M. Arunraj and D. Ahrens. "A hybrid seasonal autoregressive integrated moving average and quantile regression for daily food sales forecasting." International Journal of Production Economics, 170:321-335, 2015.

[5] S. J. Taylor and B. Letham. "Forecasting at scale." The American Statistician, 72(1):37-45, 2018. https://facebook.github.io/prophet/

[6] Chroma. "ChromaDB: The AI-native open-source embedding database." 2023. https://www.trychroma.com/

[7] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." Proceedings of EMNLP-IJCNLP, pp. 3982-3992, 2019.

[8] S. Ramirez. "FastAPI: Modern, fast web framework for building APIs with Python." 2019. https://fastapi.tiangolo.com/

[9] Kaggle. "Store Sales — Time Series Forecasting." Kaggle Competition, 2022. https://www.kaggle.com/competitions/store-sales-time-series-forecasting

[10] Amazon Web Services. "Amazon Forecast Developer Guide." AWS Documentation, 2023. https://docs.aws.amazon.com/forecast/

[11] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski. "DeepAR: Probabilistic forecasting with autoregressive recurrent networks." International Journal of Forecasting, 36(3):1181-1191, 2020.

8. Appendix

8.1 GitHub Repository

Public repository: https://github.com/usp787/CS6220_Final_Project

8.2 Installation and Setup Instructions

Prerequisites: Python 3.10+, conda (recommended) or pip.

Step 1: Clone the repository

```
git clone https://github.com/usp787/CS6220_Final_Project.git
cd CS6220_Final_Project
```

Step 2: Install dependencies

```
pip install pandas numpy scikit-learn prophet matplotlib seaborn
pip install fastapi uvicorn python-multipart
pip install chromadb sentence-transformers
```

Step 3: Generate synthetic data

```
python generate_data.py
```

Step 4: Run the forecasting pipeline

```
python pipeline.py
```

Step 5: Start the web application

```
uvicorn app:app --reload --host 0.0.0.0 --port 8000
# Open http://localhost:8000 in your browser
```

8.3 Project Structure

```
CS6220_Final_Project/
  data/                   -- Sales CSVs, weather data, SQLite database
  models/                 -- Serialized Prophet model pickles (16 files)
  output/                 -- Forecast results, reports, and figures
    figures/              -- Three matplotlib visualizations
  static/                 -- HTML frontend (index.html)
  app.py                  -- FastAPI backend (4 endpoints)
  pipeline.py             -- Pipeline orchestrator (subprocess isolation)
  rag_validator.py        -- RAG validation layer (ChromaDB + MiniLM)
  generate_data.py        -- Synthetic data generator
  step1-step9*.py         -- Individual pipeline steps
  retail_inventory_eda.ipynb  -- Exploratory data analysis notebook
  README.md               -- Project documentation
```

8.4 Use Cases and Target Users

The Retail Inventory Twin addresses the needs of several specific user groups. First, small coffee shop and bakery owners who currently rely on manual inventory counts and intuition for reorder decisions. The system provides them with a data-driven 14-day forecast and clear stock-out risk alerts (ALERT, Caution, OK) without requiring technical expertise. A shop owner can upload their point-of-sale CSV export and stock counts to receive actionable reorder guidance within minutes.

Second, regional retail chain managers overseeing 5-20 locations face the challenge of coordinating inventory across stores with different demand patterns. The weather-augmented forecasting is particularly valuable here, as stores in different microclimates may experience different demand shifts during weather transitions. The system's item-level granularity allows managers to identify which products at which locations are most at risk.

Third, supply chain analytics students and practitioners can use the system as an educational tool to understand the impact of exogenous variables on demand forecasting. The dual-model comparison (baseline vs. weather-augmented) with quantified delta MAE provides a clear demonstration of when and why external regressors improve predictions.

The system's practical value lies in reducing two concrete costs: (1) lost revenue from stock-outs, which the system mitigates through early warning alerts, and (2) waste from overstocking perishable items, which the system addresses by providing more accurate demand estimates that account for upcoming weather conditions.